

Documentation Complete — Projets 8 a 36

Objectif, outils utilises (et pourquoi), bugs corriges, et explication du code
Dossier Talent-Performer-Projects-main — projets 8 a 36

Ce document couvre les projets 8 a 36 du depot (les projets 1 a 7 sont documentes separement dans Documentation_Projets_1a7.pdf). Pour chaque projet : ce qu'il fait et pour qui, les outils utilises et la raison de chaque choix technique, les bugs trouves en essayant de le faire fonctionner (la quasi-totalite du code ciblait une version anterieure des librairies utilisees, notamment agno et mistralai, dont l'API a beaucoup change depuis), et une explication du code fichier par fichier. Les projets 12, 13, 15, 17, 19 et 28 n'existent pas dans le depot (numerotation non continue).

Sommaire

#	Projet	En une phrase	Etat
8	8-mistral_OCR	Legal Copilot : lire un document juridique et repondre a des questions dessus.	Fonctionnel
9	9-Arize	Pipeline multi-agents avec auto-evaluation independante (Arize Phoenix).	Fonctionnel
10	10-Workflow_Builder	Workflows d'automatisation no-code (n8n / Activepieces).	Fonctionnel
11	11-Notion_RAG	Chat sur une base de connaissances Notion d'entreprise.	Bloque
14	14-Multi_AgentAGNO	Equipe d'agents finance (recherche + reasoning) servie via Playground.	Bloque
16	16-Whatsapp	Bot WhatsApp gestionnaire de todo-list (Todoist).	Bloque
18	18-Memory_agent	Assistants sante / immobilier avec memoire persistante (Mem0).	Bloque
20	20-Agno-Team	Analyse de contrats par 3 agents specialises coordonnees.	Fonctionnel
21	21-SDV-synthetise	Generation de donnees synthetiques (SDV + GPT-2).	Fonctionnel
22	22-Cyber_Workflow	Reconnaissance + tests de securite passifs sur un domaine.	Fonctionnel
23	23-Whatsapp-lette	Bot WhatsApp relie a un agent Letta.	Bloque
24	24-ResearchAnalyst	Reproduit un rapport pour un autre pays a partir d'un PDF source.	Fonctionnel
25	25-PriceAndPositioning	Analyse concurrentielle de prix Amazon (recherche web IA).	Mis de cote
26	26-PlateDetection	Detection de plaques d'immatriculation + alerte SMS.	Bloque

27	27-ImportExportSpecialist	Conformite import/export UK-Maroc, verification et analyse de risque.	Fonctionnel
29	29-Investment_insight	Profil client + recommandation d'investissement.	Mis de cote
30	30-compliance_agent	Rapport de conformite reglementaire UE, export Word.	Fonctionnel
31	31-Procurement-agent	Recherche de fournisseurs pour une liste de produits.	Mis de cote
32	32-Advance-Research-agent	Assistant de recherche academique avec base de connaissances PgVector.	Pret, non teste
33	33-FreightAgent	Estimation de couts de fret (UPS/FedEx/DHL) via WhatsApp.	Corrige, a confirmer
34	34-ImportExportSpecialist	Variante autonome (CLI) du projet 27, export Word direct.	Fonctionnel
35	35-HR-Assistant	Assistant RH (conges, projets, approbations) via Airtable.	Bloque
36	36-rfp-Agent	Remplit un questionnaire d'appel d'offres via Google Sheets.	Bloque

Projet 8 — 8-mistral_OCR

Legal Copilot : OCR de document juridique + questions-reponses

Objectif

Uploader un PDF juridique (contrat, reglement...) via Streamlit, en extraire le texte avec l'OCR de Mistral (qui gere la mise en page et les tableaux, contrairement a une extraction PDF classique), puis poser des questions en langage naturel sur son contenu. Le systeme retrouve les passages les plus pertinents par recherche vectorielle avant de generer la reponse, et peut aussi produire un resume complet du document en un clic.

Outils utilises

Outil		Role dans ce projet
Mistral (mistral-ocr-latest)	OCR	Convertit le PDF en Markdown propre (titres, tableaux, paragraphes) — bien plus fiable qu'une simple extraction de texte pour un document juridique structure.
Mistral (mistral-embed)	Embed	Transforme chaque morceau (chunk) du document en vecteur numerique pour permettre une recherche par similarite de sens.
FAISS (IndexFlatL2)		Index vectoriel local en memoire : retrouve les 2 chunks les plus proches de la question sans avoir besoin d'une base de donnees externe, adapte a un document unique traite a la volee.
mistral-large-latest		Modele de chat qui redige le resume ou la reponse a partir des chunks retrouves (technique RAG : Retrieval-Augmented Generation).
chardet		Detecte automatiquement l'encodage du fichier Markdown genere par l'OCR avant de le decoder, pour eviter une erreur si l'encodage n'est pas de l'UTF-8 pur.
Streamlit		Interface : upload PDF, bouton Lancer OCR, case a cocher pour afficher le texte extrait, bouton Resume, champ de question.

Bugs trouves et corriges

- Ce projet n'utilise pas agno (appels directs au SDK Mistral) : aucun probleme de compatibilite de librairie ici. Le seul blocage etait l'absence de MISTRAL_API_KEY dans le .env — cle fournie par l'utilisateur en cours de session et ajoutee au fichier.
- Le code met deja en cache le resultat de l'OCR (ocr_meta.txt retient le chemin du dernier fichier traite) pour eviter de relancer l'OCR a chaque interaction Streamlit sur le meme PDF.

Etat final : fonctionnel — OCR, resume et questions-reponses testes avec succes sur un contrat PDF reel (ServicesAgreementSample.pdf).

legal_copilot.py

Classe OCRService : OCR, cache, embeddings, recherche vectorielle et generation de reponse.

Lignes 19-43

```
def ocr_pdf(self, file_path):
    uploaded_pdf = self.client.files.upload(
        file={"file_name": file_path.split("/")[-1], "content": open(file_path, "rb")},
        purpose="ocr"
    )
    signed_url = self.client.files.get_signed_url(file_id=uploaded_pdf.id)

    ocr_response = self.client.ocr.process(
        model="mistral-ocr-latest",
        document={"type": "document_url", "document_url": signed_url.url},
        include_image_base64=True
    )

    with open("ocr_response.md", "w", encoding="utf-8") as f:
        f.write("\n".join([page.markdown for page in ocr_response.pages]))

    with open("ocr_meta.txt", "w") as meta:
        meta.write(file_path)
```

Le PDF est d'abord televerse vers l'API Mistral (purpose="ocr"), qui renvoie un identifiant de fichier. On recupere une URL signee temporaire pour ce fichier, puis on demande a mistral-ocr-latest de le traiter page par page. Chaque page renvoyee contient son propre Markdown, qu'on concatene dans ocr_response.md. Le chemin du fichier source est note dans ocr_meta.txt : c'est ce qui permet a ocr_is_valid() de savoir si le cache correspond toujours au PDF actuellement uploadé, sans refaire l'OCR inutilement.

Lignes 71-102

```
def process_question(self, question, file_path):
    text = self.read_markdown_file("ocr_response.md")
    chunk_size = 2048
    chunks = [text[i:i + chunk_size] for i in range(0, len(text), chunk_size)]

    text_embeddings = []
    for chunk in chunks:
        embedding = self.get_text_embedding(chunk)
        text_embeddings.append(embedding)
        time.sleep(2)

    text_embeddings = np.array(text_embeddings)
    question_embedding = np.array([self.get_text_embedding(question)])

    d = text_embeddings.shape[1]
    index = faiss.IndexFlatL2(d)
    index.add(text_embeddings)

    D, I = index.search(question_embedding, k=2)
    retrieved_chunk = [chunks[i] for i in I.tolist()[0]]

    prompt = f"""Context information is below.
-----
{retrieved_chunk}
-----
Given the context information and not prior knowledge, answer the query.
Query: {question}
Answer: """""

    return self.run_mistral(prompt)
```

Coeur du RAG (Retrieval-Augmented Generation) : le document est decoupe en morceaux de 2048 caracteres, chacun est transforme en vecteur (avec une pause de 2 secondes entre chaque appel pour respecter le debit de

l'API Mistral), puis indexe dans FAISS. La question de l'utilisateur est elle aussi vectorisée, et FAISS retrouve les 2 chunks dont le vecteur est le plus proche (distance L2) de celui de la question. Seuls ces 2 chunks — pas le document entier — sont insérés dans le prompt final envoyé au modèle de chat, ce qui garde le prompt court même pour un long contrat.

app.py

Interface Streamlit : upload, OCR, affichage, resume, questions.

Lignes 12-25

```
if uploaded_file:
    os.makedirs("pdf", exist_ok=True)
    file_path = f"pdf/{uploaded_file.name}"
    with open(file_path, "wb") as f:
        f.write(uploaded_file.read())

    if st.button(" Run OCR"):
        try:
            with st.spinner("OCR en cours..."):
                ocr_service.ocr_pdf(file_path)
            ocr_performed = True
        except Exception as e:
            st.error(f"Error during OCR: {e}")
```

Le PDF uploadé est d'abord sauvegardé sur disque (dans pdf/) avant d'être envoyé à l'OCR — l'API Mistral attend un chemin de fichier ouvrable, pas les octets bruts reçus par Streamlit. L'OCR n'est déclenché qu'au clic du bouton, pas automatiquement à l'upload, pour laisser le contrôle à l'utilisateur (l'OCR consomme des crédits API).

Projet 9 — 9-Arize

Pipeline multi-agents (Recherche -> Analyse -> Evaluation) avec auto-evaluation independante

Objectif

Analyser un sujet de politique publique (ici : les politiques agricoles britanniques et leur impact sur les communes agricoles) via un pipeline de 3 agents specialises qui se relaient : un agent Recherche collecte des donnees recentes sur le web, un agent Analyse les synthetise en rapport structure, et un agent Evalueur juge si ce rapport est « impactant ». Un **second** modele, totalement independant de l'equipe (technique « LLM-as-judge » inspiree d'Arize Phoenix), rejuge ensuite le rapport final de maniere impartiale — utile pour detecter si un agent se note lui-meme trop genereusement. Disponible en ligne de commande et en interface Streamlit avec suivi de progression en direct.

Outils utilises

Outil	Role dans ce projet
<code>agno.team.Team</code>	Orchestre les 3 agents en leur delegant reellement chaque etape (Recherche puis Analyse puis Evaluation) — remplace le pattern « agent comme outil » qui n'existe plus dans la version installee d'agno.
Groq (gpt-oss-120b / gpt-oss-20b)	gpt-oss-120b pour les 3 agents specialises (qualite de redaction) ; gpt-oss-20b, plus leger, pour le leader du Team qui doit relire/synthetiser en un seul appel les rapports combines des 2 premiers agents (quota Groq plus permissif sur ce modele).
DuckDuckGoTools	Recherche web pour l'agent Recherche — remplace GoogleSearchTools, retire de la librairie agno.
arize-otel + openinference	Envoie les traces d'execution (OpenTelemetry) vers la plateforme Arize pour observabilite — rendu optionnel : le pipeline tourne meme sans compte Arize configure.
phoenix.evals (llm_classify)	Fait juger le rapport final par un modele independant de l'equipe qui l'a redige, avec un prompt d'evaluation dedie — pattern « LLM-as-judge » pour une evaluation moins biaisee qu'un agent qui s'auto-evalue.
pandas	Structure les donnees pour phoenix.evals et exporte les resultats d'evaluation en CSV.
Streamlit	Interface avec barre de progression mise a jour en temps reel (via le streaming d'evenements du Team) montrant quel agent travaille actuellement.

Bugs trouves et corriges

- **Pattern « agent comme outil » obsolete** : le script original passait 3 agents directement dans `tools=[...]` d'un agent « manager », en esperant qu'il les appelle. Dans agno 2.8.7, un agent ne delegue plus automatiquement a d'autres agents passes en outils — il se contentait d'halluciner un faux plan sans jamais les executer. Remplace par `agno.team.Team(members=[...])`, le mecanisme officiel de delegation.
- **GoogleSearchTools retire** de la librairie agno remplace par `DuckDuckGoTools`.

- **success_criteria retire** du constructeur de Team replie dans le champ `instructions` de l'agent concerne.
- **Depassement de quota Groq (TPM)** : l'agent Recherche accumulait les resultats bruts de plusieurs recherches web (souvent 20+ minutes sans jamais conclure), et le Team devait ensuite relire d'un coup les rapports complets de Recherche + Analyse pour les synthetiser — les deux dépassaient la limite Groq (8000 tokens/minute en tier gratuit). Corrige avec `compress_tool_results=True`, `tool_call_limit=6` sur l'agent Recherche, et le modele plus léger gpt-oss-20b pour le leader du Team.
- `register()` d'arize-otel leve une `ValueError` si `ARIZE_SPACE_ID`/`ARIZE_API_KEY` sont vides le tracing Arize est rendu optionnel (le pipeline continue sans lui si ces variables sont absentes).
- Plantage console Windows (cp1252) sur des caracteres Unicode presents dans les logs internes d'agno corrige par la reconfiguration UTF-8 de `stdout/stderr` en tete de script.
- Le script original melangeait logique metier et interface en un seul fichier refactorise en un moteur partage `policy_pipeline.py` (agents, Team, appels API), reutilise a la fois par `evaluation_agent.py` (CLI) et `app.py` (Streamlit), pour ne pas dupliquer la definition des agents entre les deux.

Etat final : fonctionnel — pipeline complet teste en CLI et en Streamlit, avec export CSV des resultats d'evaluation.

policy_pipeline.py

Moteur partage : definition des 3 agents, le Team, et l'auto-evaluation Phoenix.

Lignes 225-257

```
AgentGestionnaire = Team(
    name="AgentGestionnaire",
    model=Groq(id="openai/gpt-oss-20b", api_key=GROQ_API_KEY),
    members=[AgentRecherche, AgentAnalyse, AgentEvalueur],
    description="Un agent orchestrateur coordonnant le flux d'evaluation complet...",
    instructions=dedent("""
        Etapes :
        1. Appeler AgentRecherche pour rassembler le contexte pertinent.
        2. Transmettre le contexte brut a AgentAnalyse pour produire une analyse structuree.
        3. Demander a AgentEvalueur d'evaluer l'impact du rapport politique.
        Ne pas generer de texte vous-meme. Router chaque tache vers l'agent correct.
    """),
    markdown=True,
    debug_mode=True,
    compress_tool_results=True,
)
```

Le Team recoit les 3 agents specialises comme members et un jeu d'instructions qui decrit l'ordre des etapes — c'est le Team lui-meme qui decide, via son propre modele (le « leader »), quand deleguer a quel membre en generant des appels d'outil internes (`delegate_task_to_member`). L'instruction explicite « ne pas generer de texte vous-meme » evite que le leader ne redige sa propre reponse au lieu de deleguer.

Lignes 266-287

```
def run_multi_agent_pipeline(saisie_utilisateur, progress_callback=None):
    final_content = None
    for event in AgentGestionnaire.run(saisie_utilisateur, stream=True, stream_events=True):
        tool = getattr(event, "tool", None)
        if tool is not None and tool.tool_name == "delegate_task_to_member" and progress_callback:
            member_id = (tool.tool_args or {}).get("member_id", "")
            progress_callback(MEMBER_LABELS.get(member_id, f"{member_id} en cours..."))

        if getattr(event, "event", None) == "TeamRunCompleted" and getattr(event, "content",
None):
            final_content = event.content

    return final_content or "(Aucune reponse generee.)"
```

En streamant les evenements (`stream_events=True`) plutot que d'attendre la reponse finale, on peut intercepter chaque appel `delegate_task_to_member` pour savoir en temps reel quel agent est en train de travailler et mettre a jour la barre de progression Streamlit. L'evenement `TeamRunCompleted` (avec le prefixe « Team ») n'est emis que par le Team lui-meme, jamais par un membre delegue — ce qui suffit a isoler la reponse finale synthetisee des reponses intermediaires de chaque agent.

Lignes 311-343

```

def run_auto_evaluation(saisie_utilisateur, contenu_final):
    donnees_reponse = {
        "input": [saisie_utilisateur],
        "actual_output": [contenu_final],
        "expected_output": [AgentAnalyse.expected_output],
    }
    df_reponse = pd.DataFrame(donnees_reponse)
    df_reponse["formatted_prompt"] = df_reponse.apply(format_template, axis=1)

    return llm_classify(
        dataframe=df_reponse,
        template=MODELE_EVAL_ROUTER,
        model=OpenAIModel(api_key=GROQ_API_KEY, base_url="https://api.groq.com/openai/v1",
model=GROQ_MODEL_ID),
        rails=["not impactful", "impactful"],
        provide_explanation=True,
        concurrency=1,
    )

```

C'est l'etape d'auto-evaluation independante : `llm_classify` (de la librairie Phoenix d'Arize) prend l'entree, la sortie reelle du pipeline et la sortie attendue, les insere dans un prompt de jugement dedie, et force le modele a repondre par l'une des deux etiquettes de rails. C'est un jugement distinct de celui d'AgentEvalueur (qui fait partie du meme Team que les agents qu'il evalue) — utile pour verifier que l'equipe ne se donne pas un satisfecit trop facilement. `OpenAIModel` pointe ici vers l'endpoint compatible OpenAI de Groq plutot que vers OpenAI lui-meme.

Projet 10 — 10-Workflow_Builder

Workflows d'automatisation no-code (n8n / Activepieces) pour la gestion clients

Objectif

Construire, sans écrire de backend, des workflows d'automatisation pour un scénario CRM complet : qualification automatique des leads entrants (formulaire -> score -> routage), sequences d'emails d'onboarding programmées pour les nouveaux clients, envoi d'enquetes de satisfaction apres un service termine, et relance des factures echues. Chaque workflow est un graphe de « noeuds » (declencheur, code, condition, appel HTTP...) decrit en JSON et importe dans un outil no-code (n8n ou Activepieces) qui l'execute visuellement.

Outils utilises

Outil	Role dans ce projet
n8n (via npx n8n)	Outil d'automatisation open-source qui execute des workflows definis en JSON (noeuds + connexions) ; installe localement sans Docker.
Activepieces	Alternative no-code plus recente, deployee ici en conteneurs Docker (app + Postgres + Redis) pour une stack complete et isolee.
Docker Compose	Orchestre les 3 services Activepieces avec les cle de chiffrement/JWT requises, generees pour l'occasion.
Serveur HTTP local (python -m http.server)	Sert temporairement le fichier JSON du workflow pour utiliser la fonction « Import from URL » de n8n, en contournement du dialogue natif de selection de fichier (non automatisable dans le navigateur en bac a sable).

Bugs trouves et corriges

- gestionclient.json etait corrompu : le dernier noeud du fichier etait tronque (il manquait id, name, type, typeVersion et position), et la cle connections — qui relie les noeuds entre eux pour former le graphe — etait totalement absente du fichier. Reconstitue noeud par noeud et connexion par connexion, en deduisant la logique des branches (qualification chaude/tiede/froide, sequences cron) a partir des positions et noms des noeuds existants.
- Le dialogue natif d'upload de fichier du navigateur n'est pas pilotable par les outils d'automatisation du navigateur sandboxe contourne en servant le JSON via un petit serveur HTTP local et en utilisant l'option « Import from URL » de n8n a la place de « Import from file ».
- Un simple docker run pour Activepieces echouait (variables d'environnement obligatoires manquantes) remplace par un docker-compose.activepieces.yml complet avec AP_ENCRYPTION_KEY, AP_JWT_SECRET et AP_WORKER_TOKEN generes.

Etat final : fonctionnel — workflow importe et visualise dans n8n ; stack Activepieces demarree via Docker Compose (creation du compte utilisateur/organisation laissee volontairement a l'utilisateur, cf. regles de securite sur les formulaires de creation de compte).

gestionclient.json

Graphe de noeuds n8n : 4 sous-flux (leads, onboarding, satisfaction, factures).

Lignes 19-31

```
{
  "parameters": {
    "jsCode": "const data = $input.first().json;\nlet score = 0;\nconst weights = {\n  company_size: {'startup': 2, 'pme': 5, 'enterprise': 10},\n  budget: {'low': 1, 'medium': 5, 'high': 10},\n  urgency: {'low': 1, 'medium': 3, 'high': 7},\n  ...\n};\nObject.keys(weights).forEach(c => { if (data[c]) score += weights[c][data[c]]; });\nlet qualification = score >= 35 ? 'hot' : score >= 20 ? 'warm' : 'cold';\nreturn {json: {...data, lead_score: score, qualification_status: qualification}};"
  },
  "id": "lead-scoring",
  "name": "Calcul Score Lead",
  "type": "n8n-nodes-base.code",
  "typeVersion": 2,
  "position": [460, 300]
}
```

Un noeud `n8n-nodes-base.code` execute du JavaScript arbitraire dans le workflow : ici, il calcule un score pondere a partir des criteres du lead (taille d'entreprise, budget, urgence, decideur ou non, adequation sectorielle) et en deduit une categorie (chaud/tiede/froid). Ce noeud recoit les donnees du webhook precedent en entree et les enrichit avant de les transmettre au noeud suivant.

Lignes 32-72

```
{
  "parameters": {
    "conditions": {
      "conditions": [
        {
          "leftValue": "={{ $json.qualification_status }}",
          "rightValue": "hot",
          "operator": "equals",
          "type": "string",
          "operation": "equals"
        },
        {
          "leftValue": "={{ $json.qualification_status }}",
          "rightValue": "warm",
          "operator": "equals",
          "type": "string",
          "operation": "equals"
        }
      ],
      "combinator": "or"
    }
  },
  "id": "qualification-switch",
  "name": "Routage par Qualification",
  "type": "n8n-nodes-base.switch"
}
```

Un noeud switch route ensuite le lead vers des branches differentes du workflow selon la valeur calculee par le noeud precedent : un lead « hot » ou « warm » declenche la creation immediate dans le CRM et une notification Slack au commercial ; un lead « cold » part vers une sequence d'automation marketing plus legere. C'est la cle connections du fichier (absente dans la version corrompue) qui relie ce noeud aux noeuds suivants selon la branche empruntee.

Projet 11 — 11-Notion_RAG

Chat en ligne de commande sur une base de connaissances Notion d'entreprise

Objectif

Assistant en ligne de commande permettant aux employes de poser des questions en langage naturel a une base de connaissances interne stockee dans une base de donnees Notion (paires Question/Reponse, avec departement et tags). L'agent interroge Notion a chaque question, trouve les entrees les plus proches de la question posee, et repond en citant la source.

Outils utilises

Outil	Role dans ce projet
<code>notion_client</code> (Client)	SDK officiel Notion : interroge (<code>databases.query</code>) toutes les pages de la base de connaissances configuree.
<code>agno.agent.Agent</code> + <code>knowledge_retriever</code>	Au lieu d'une base vectorielle classique, le « retrieveur » est ici une fonction Python maison qui interroge directement l'API Notion a chaque question — pas de reindexation necessaire quand la base Notion change.
Groq (<code>openai/gpt-oss-120b</code>)	Remplace le modele OpenAI (<code>gpt-4o</code>) du script original, pour reutiliser la cle Groq deja disponible.
Correspondance mots-cles par	Pas d'embeddings ici : la fonction compare l'ensemble des mots de la question a l'ensemble des mots de chaque paire question/reponse Notion, et retient les correspondances a 60% ou plus — une approche simple mais suffisante pour une petite FAQ d'entreprise.

Bugs trouves et corriges

- `retriever=` renomme `knowledge_retriever=` dans les versions recentes d'agno (meme signature de fonction) — seule modification necessaire cote code.
- Le reste de la logique (gestion des variantes de casse des proprietes Notion : `Question/question`, `Response/response/answer...`) etait deja ecrit de maniere defensive dans le script original.

Etat : bloque — necessite un NOTION_TOKEN (integration Notion) et un NOTION_DATABASE_ID pointant vers une base de connaissances Notion reelle. La creation d'une integration/d'un compte Notion a ete volontairement laissee a l'utilisateur (regle de securite : ne jamais remplir de formulaire de connexion ou de creation de compte). Code corrige mais non teste en conditions reelles faute d'accès a un espace Notion configure.

noton_rag.py

Classe NotionRAG : agent agno avec retrieveur Notion personnalis .

Lignes 17-28

```
def _create_agent(self):
    return Agent(
        model=Groq(id="openai/gpt-oss-120b", api_key=GROQ_API_KEY),
        knowledge=None,
        search_knowledge=True,
        knowledge_retriever=self._notion_retriever
    )
```

Contrairement aux autres projets qui utilisent une base vectorielle (LanceDB, PgVector...), cet agent n'a pas de knowledge pre-index e : `search_knowledge=True` avec `knowledge_retriever` pointant vers une m thode maison suffit   agno pour appeler cette fonction   chaque question et lui fournir les documents retrouv s comme contexte.

Lignes 60-68

```
question_words = set(question.lower().split())
answer_words = set(answer.lower().split())
all_words = question_words.union(answer_words)

matches = len(query_words.intersection(all_words))
match_ratio = matches / len(query_words) if query_words else 0

if match_ratio >= 0.6: # 60% match threshold
    ...
```

Pour chaque entree de la base Notion, on calcule la proportion de mots de la question de l'utilisateur qui se retrouvent dans la question OU la reponse stock e. Au-del  de 60% de recouvrement, l'entree est consid r e comme pertinente et renvoy e   l'agent, tri e par score de correspondance d croissant. C'est une m thode de recherche lexicale simple (pas s mantique) : elle rate les reformulations qui n'utilisent aucun mot commun avec l'entree Notion, mais reste rapide et sans d pendance   un embedder.

Projet 20 — 20-Agno-Team

Analyse de contrats par une equipe de 3 agents specialises

Objectif

Permettre a l'utilisateur d'uploader un contrat (PDF, DOCX ou TXT) via Streamlit et d'obtenir une analyse consolidee en quatre parties : resume executif, contexte juridique (clauses problematiques citees), retours de structuration du document, et strategies de negociation. Trois agents specialises (Structure, Juridique, Negociation) analysent chacun le contrat sous leur angle propre, et un quatrieme agent — le *Team* — coordonne leur travail et fusionne leurs contributions en un rapport unique et coherent.

Outils utilises

Outil	Role dans ce projet
Streamlit	Interface d'upload et d'affichage du rapport final.
<code>agno.team.Team</code>	Orchestre les 3 agents specialises et synthetise leurs sorties.
Groq (openai/gpt-oss-120b / 20b)	Modele de langage des 3 agents specialises et du chef d'equipe.
PyMuPDF (fitz)	Extrait le texte brut des fichiers PDF et DOCX uploades.

Bugs trouves et corriges

- **API agno obsolete** : `show_tool_calls=` n'existe plus sur `Agent()` ni `Team()` — renomme `debug_mode=`.
- `success_criteria=` **supprime de** `Team()` : ce parametre n'existe plus dans la version installee d'agno. Son contenu (les criteres de reussite du rapport final) a ete fusionne directement dans `instructions=`.
- **Pas de cle OpenAI disponible** : le code original utilisait `OpenAIChat(id='gpt-4o')` pour les 4 agents ; remplace par Groq.
- **Le chef d'equipe demandait le contrat au lieu de deleguer** : sans instruction explicite, le modele repondait « fournissez-moi le texte du contrat » au lieu d'appeler ses 3 agents membres (qui, eux, ont acces a l'outil `get_document()` pour recuperer le contrat deja charge en memoire). Corrige en ajoutant `delegate_to_all_members=True` et une instruction explicite interdisant de demander le texte a l'utilisateur.
- **Depassement du quota Groq (8000 tokens/minute) lors de la synthese finale** : combiner les 3 rapports complets en un appel unique depassait la limite du modele 120B ; le chef d'equipe utilise desormais le modele 20B, plus leger et avec un quota superieur, pour cette etape.

Etat final : fonctionnel — rapport complet en 4 parties genere avec succes sur un contrat reel.

utils.py

Extraction du texte du document uploadé (variable globale partagée).

Lignes 1-44

```
current_file_path = None

def set_document_file(file_path):
    global current_file_path
    current_file_path = file_path

def get_document():
    global current_file_path
    if not current_file_path:
        return {"error": "Aucun fichier", "content": ""}
    file_path = Path(current_file_path)
    extension = file_path.suffix.lower()
    if extension == '.pdf':
        content = extract_pdf(file_path)
    elif extension in ['.docx', '.doc']:
        content = extract_word_simple(file_path)
    elif extension == '.txt':
        content = extract_text(file_path)
    else:
        return {"error": "Format non supporté", "content": ""}
    return {"content": content, "error": None}

def extract_pdf(file_path):
    doc = fitz.open(file_path)
    text = ""
    for page in doc:
        text += page.get_text()
    doc.close()
    return text
```

Le chemin du fichier à analyser est stocké dans une variable globale (`current_file_path`) plutôt que d'être passé en paramètre : c'est ce qui permet à `get_document()` d'être appelée **sans argument** par les agents, comme un outil ordinaire. L'extraction elle-même repose entièrement sur PyMuPDF, qui gère aussi bien les PDF que — de manière moins fiable — les fichiers Word.

contract_analysis.py

Les 3 agents spécialisés + le Team qui les coordonne.

Lignes 53-96

```
def create_legal_agent():
    return Agent(
        model=Groq(id=GROQ_MODEL_ID, api_key=GROQ_API_KEY),
        name='Agent Juridique',
        role='Analyste des Questions Juridiques',
        instructions=dedent("""
            Utilisez l'outil get_document() pour accéder au texte complet du contrat.
            Pour chaque problème juridique ou observation, vous DEVEZ :
            - Citer la clause, phrase ou paragraphe exact du contrat...
            - Commencer une nouvelle ligne avec 'Problème :' suivi d'une explication...
            """),
        tools=[get_document],
        debug_mode=False,
        markdown=True,
    )
```

Chaque agent reçoit la même fonction `get_document` comme outil, mais des instructions très différentes qui orientent son analyse (structure, droit, ou négociation). L'agent juridique est le plus strict : il DOIT citer le texte exact du contrat pour chaque problème soulevé, afin d'éviter les affirmations non sourcées.

Lignes 144-188

```
manager_agent = Team(  
    members=[structure_agent, legal_agent, negotiate_agent],  
    model=Groq(id="openai/gpt-oss-20b", api_key=GROQ_API_KEY),  
    mode='coordinate',  
    delegate_to_all_members=True,  
    instructions=dedent("""  
        Critere de reussite : un resume bien organise et tracable du contrat...  
        IMPORTANT : Ne demandez JAMAIS a l'utilisateur de fournir le texte du contrat.  
        Deleguez immediatement la tache aux trois agents ci-dessus...  
  
        Format final :  
        1. Resume Executif  
        2. Contexte Juridique (Avec clauses citees et explications des problemes)  
        3. Retours sur la Structure du Contrat  
        4. Recommandations de Negociation  
        """),  
    debug_mode=True,  
    markdown=True,  
)
```

Le Team en mode coordinate agit comme un chef de projet : il envoie une tache a chacun des 3 agents membres, attend leurs reponses, puis redige lui-meme le rapport final en suivant le plan impose dans ses instructions. `delegate_to_all_members=True` force cette delegation systematique plutot que de laisser le modele decider, au cas par cas, s'il delegue ou non — un comportement peu fiable observe lors des tests.

Projet 21 — 21-SDV-synthetise

Generation de donnees synthetiques (tabulaires et texte narratif)

Objectif

Outil en ligne de commande (menu interactif) pour generer des donnees synthetiques a partir d'un jeu de donnees e-commerce reel : soit des lignes tabulaires statistiquement proches de l'original (deux modes — *replication de schema* ou *realisme et variete*), soit des champs narratifs (messages de support client) generes par un modele de langage local (GPT-2). Des rapports de diagnostic et de qualite permettent ensuite de verifier que les donnees generees ressemblent statistiquement aux vraies.

Outils utilises

Outil	Role dans ce projet
SDV (GaussianCopulaSynthesizer)	Apprend la distribution statistique des vraies donnees et genere de nouvelles lignes plausibles.
sdv.evaluation	Rapports de diagnostic et de qualite comparant donnees reelles et synthetiques.
transformers (pipeline GPT-2)	Genere du texte narratif libre (messages de support fictifs), localement, sans API.
pandas	Chargement et sauvegarde des jeux de donnees CSV.

Bugs trouves et corriges

- **Chemin de fichier casse** : le CSV source etait lu via un chemin relatif au depot entier (21-SDV-synthetise/Data/...) alors que les fichiers de sortie etaient ecrits avec un chemin relatif au dossier du script (Data/...) — incoherent, et cassait des que le script n'etait pas lance depuis la racine exacte du depot. Corrige en resolvant tous les chemins par rapport au dossier du script lui-meme.
- **Conflit TensorFlow / protobuf** : importer transformers (pour le pipeline GPT-2) declenche le chargement de TensorFlow en plus de PyTorch, et la version de TensorFlow installee est incompatible avec la version de protobuf presente sur la machine — crash au demarrage, avant meme d'utiliser la fonctionnalite narrative. Corrige avec `os.environ['USE_TF'] = '0'` avant l'import.
- **Crash d'encodage Windows** : le message de succes contenant un emoji (👉) faisait planter `print()` sur la console Windows par default (cp1252). Corrige en reconfigurant `sys.stdout/stderr` en UTF-8 des le debut du script.

Etat final : fonctionnel — generation de donnees synthetiques testee de bout en bout (mode *replication de schema*).

data_synthesizer.py

Menu interactif : synthese tabulaire, donnees narratives, rapports de qualite.

Lignes 1-16

```
import sys
for _stream in (sys.stdout, sys.stderr):
    if hasattr(_stream, "reconfigure"):
        _stream.reconfigure(encoding="utf-8", errors="replace")

import pandas as pd
import warnings, os
os.environ.setdefault("USE_TF", "0")

from sdv.metadata import Metadata
from sdv.single_table import GaussianCopulaSynthesizer
from sdv.evaluation.single_table import run_diagnostic, evaluate_quality, get_column_plot
from transformers import pipeline

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(SCRIPT_DIR, "Data")
os.makedirs(DATA_DIR, exist_ok=True)
real_data = pd.read_csv(os.path.join(DATA_DIR, "E-Commerce Dataset.csv"), nrows=500)
```

Les deux correctifs (encodage UTF-8, desactivation de TensorFlow) doivent impérativement s'exécuter **avant** tout autre import qui déclencherait le problème — d'où leur position tout en haut du fichier. SCRIPT_DIR ancre tous les chemins de données sur l'emplacement réel du fichier, quel que soit le répertoire de travail courant.

Lignes 23-46

```
def synthesize_data(realism=False):
    global metadata, synthetic_data
    metadata = Metadata.detect_from_dataframe(real_data)
    synthesizer = GaussianCopulaSynthesizer(metadata)
    synthesizer.fit(real_data)
    synthetic_data = synthesizer.sample(num_rows=500)
    synthetic_data.to_csv(os.path.join(DATA_DIR, "synthetic_data.csv"), index=False)
    print("\nDonnées synthétiques sauvegardées dans Data/synthetic_data.csv")
```

Metadata.detect_from_dataframe devine automatiquement le type de chaque colonne (numérique, catégorielle, date...). Le GaussianCopulaSynthesizer apprend ensuite les corrélations statistiques entre colonnes et génère de nouvelles lignes qui respectent ces mêmes corrélations, sans jamais recopier une ligne réelle telle quelle.

Projet 22 — 22-Cyber_Workflow

Reconnaissance et tests de securite passifs sur un domaine

Objectif

Etant donne un nom de domaine, executer une reconnaissance passive (WHOIS, DNS, en-tetes HTTP) puis un ensemble de verifications de securite non intrusives (en-tetes de securite manquants, repertoires sensibles exposes, fichier .env accessible, redirection HTTPS, etc.), et produire un rapport avec un score de risque global et des recommandations. Deux agents cooperent : un agent de *reconnaissance* qui categorise la cible et recommande quels tests lancer, et un agent de *test* qui choisit et execute uniquement les outils pertinents parmi ceux recommandes.

Outils utilises

Outil	Role dans ce projet
<code>agno.workflow.Workflow</code>	Structure le pipeline en deux etapes sequentielles (recon puis test).
Groq (openai/gpt-oss-120b)	Modele des deux agents.
<code>python-whois</code>	Recupere les informations d'enregistrement WHOIS du domaine.
<code>dnspython</code>	Resout les enregistrements DNS (adresses IP) du domaine.
<code>requests</code> / <code>BeautifulSoup</code>	Recuperent et analysent les pages/en-tetes HTTP (robots.txt, sitemap, JS...).
<code>rich</code>	Affiche le rapport final formate dans le terminal.

Bugs trouves et corriges

- **API agno obsolete** : `RunResponse` n'existe plus dans `agno.agent` — renomme `RunOutput` (importe ici sous l'alias `RunResponse` pour minimiser les changements ailleurs dans le fichier).
- **Pas de cle OpenAI** : les 2 agents utilisaient `OpenAIChat(id='gpt-4o')` ; remplace par Groq.
- **Dependances manquantes** : `python-whois` et `dnspython` n'etaient pas installes.
- **Crash d'encodage Windows** : le rendu final via `rich` (bordures de tableau, icones) plantait sur la console Windows cp1252. Corrige en reconfigurant `stdout/stderr` en UTF-8.

Etat final : fonctionnel — teste sur le domaine `reserve.example.com`, rapport complet avec score de risque genere sans erreur.

offCyberTest.py

Le Workflow en 2 etapes : recon puis tests de securite.

Lignes 44-70

```
class OffensiveCybersecurityTester(Workflow):
    recon_agent: Agent = Agent(
        name='Recon Agent',
        model=Groq(id='openai/gpt-oss-120b', api_key=GROQ_API_KEY),
        instructions=dedent("""
            1. Analyser le WHOIS, DNS, et metadonnees disponibles.
            2. Categoriser la cible (e-commerce, blog, SaaS, API, portail de connexion...).
            3. Identifier les observations notables ou suspectes.
            4. Suggester des types de tests specifiques a mener ensuite.
            Ne pas inventer de donnees. N'utiliser que ce qui est fourni en entree.
        """),
    )
```

L'agent de reconnaissance ne recoit aucun outil : il travaille uniquement sur le texte brut (WHOIS, DNS, en-tetes) qui lui est fourni dans le prompt, et se contente de l'*analyser* pour orienter la suite — une separation claire entre collecte de donnees (faite en Python classique) et interpretation (faite par le modele).

Lignes 72-136

```
test_agent: Agent = Agent(
    name='Test Agent',
    model=Groq(id='openai/gpt-oss-120b', api_key=GROQ_API_KEY),
    instructions=dedent("""
        Vous etes equipe de 9 outils surs et legaux...
        1. Lire la sortie de l'agent Recon.
        2. Selectionner uniquement les outils pertinents.
        3. Executer des tests non intrusifs avec ces outils.
        Regles : ne jamais tester de maniere destructive. Seule une analyse
        publique, passive et educative est autorisee.
    """),
    tools=[check_security_headers, check_common_directories, find_js_urls,
           check_robots_txt, check_sitemap, check_env_exposure,
           check_server_header, check_http_redirect, check_github_mentions],
)

def run(self, domain: str) -> Iterator[RunResponse]:
    domain = extract_domain(domain)
    full_context = f"""Domain: {domain}\nWHOIS info: {get_whois(domain)}\n
        DNS records: {get_dns_records(domain)}\nHTTP headers:
        {get_http_headers(domain)}"""
    recon_response = self.recon_agent.run(full_context)
    yield from self.test_agent.run(recon_response.content, stream=True)
```

Le contraste avec l'agent de reconnaissance est volontaire : l'agent de test recoit 9 fonctions Python comme `tools` (definies dans `tools.py`) et ses instructions lui interdisent explicitement d'utiliser un outil qui n'a pas ete recommande par la phase de reconnaissance. Le `Workflow.run()` enchaîne les deux appels : d'abord bloquant (recon), puis en streaming (test), et transmet le texte produit par le premier agent comme entree du second — c'est cette chaine qui fait office de « memoire » entre les deux etapes.

Projet 27 — 27-ImportExportSpecialist

Conformite import/export UK-Maroc : spécialiste, verification, analyse de risque

Objectif

Etant donne une description de produit et d'expedition (type, quantite, matieres, trajet Royaume-Uni vers Maroc), produire un rapport de conformite complet : licences d'exportation requises, restrictions, taxes, permis d'importation, TVA, documents douaniers — en s'appuyant sur les textes de loi officiels (OCR'ises depuis des PDF) et, si besoin, une recherche web. Le rapport est ensuite audite par un second agent (verification) puis analyse sous l'angle des risques juridiques, operationnels et financiers par un troisieme.

Outils utilises

Outil	Role dans ce projet
Streamlit	Interface de saisie et d'affichage du rapport en 3 parties.
Mistral OCR	Convertit les PDF de loi (UK/Maroc) en Markdown exploitable.
Groq (openai/gpt-oss-120b / 20b)	4 agents : traduction, specialiste conformite, verification, analyse de risque.
DuckDuckGoTools	Recherche web complementaire si les documents de loi ne suffisent pas.

Bugs trouves et corriges

- **Import mistralai casse** : `from mistralai import Mistral` ne fonctionne plus avec la version installee (2.9.3, qui a restructure le SDK) ; corrige en `from mistralai.client import Mistral`.
- **retrieve= obsolete, et utilise avec une liste** : le code passait `retrieve=[]` a la construction de l'agent, puis reassignait `agent.retrieve = [fonction1, fonction2]` apres coup — mais le parametre s'appelle desormais `knowledge_retrieve=` et n'accepte qu'une seule fonction, pas une liste. Corrige en creant une fonction combinee qui interroge les deux documents et retourne leurs deux contenus dans une liste.
- **GoogleSearchTools n'existe plus** dans `agno` — remplace par `DuckDuckGoTools`.
- **Pas de cle OpenAI** : les 4 agents utilisaient `OpenAIChat` ; remplaces par `Groq`.
- **Depassement massif du quota Groq (jusqu'a 63 000 tokens demandes en un seul appel)** : le `retrieve` combine renvoyait les **documents de loi complets** (77 Ko et 106 Ko, soit ~180 Ko a eux deux) a chaque tour de conversation, y compris apres plusieurs recherches web qui accumulaient encore du contexte. Corrige en tronquant chaque document a 4000 caracteres et en utilisant le modele `Groq 20B`, plus leger.

Etat final : logique validee de bout en bout (rapport de conformite complet genere sur un cas test) — derniere execution de confirmation interrompue par l'epuisement du quota Groq du jour.

import_export_agent.py

OCR des lois, retrieveur combine, et les 3 agents d'analyse.

Lignes 40-57

```
def export_law_document(agent, query, num_documents=None, **kwargs):
    with open("Documents/uk-export-law.md", "r") as f:
        return f.read()

def import_law_document(agent, query, num_documents=None, **kwargs):
    with open("Documents/morocco-import-law.md", "r") as f:
        return f.read()

MAX_LAW_EXCERPT_CHARS = 4000

def combined_law_documents(agent, query, num_documents=None, **kwargs):
    return [
        export_law_document(agent, query)[:MAX_LAW_EXCERPT_CHARS],
        import_law_document(agent, query)[:MAX_LAW_EXCERPT_CHARS],
    ]
```

Ces trois fonctions ignorent toutes la query recue en parametre — elles ne font pas de recherche semantique, elles renvoient toujours l'integralite (ou un extrait) du fichier. C'est un « retrieveur » tres simple, suffisant pour un prototype mais qui perd toute pertinence ciblee par rapport a une vraie recherche vectorielle.

Lignes ~130-140

```
specialist_agent = Agent(
    name="Specialist Agent",
    model=Groq(id="openai/gpt-oss-20b", api_key=groq_key),
    instructions=dedent("""..."""),
    knowledge=None,
    search_knowledge=True,
    knowledge_retriever=combined_law_documents,
    tools=[DuckDuckGoTools()],
    compress_tool_results=True,
)
```

L'agent spécialiste combine deux sources de contexte : le retrieveur (extraits des 2 lois) et la recherche web (pour combler les lacunes). `compress_tool_results=True` demande a agno de resumer/compresser les resultats de recherche web avant de les reinjecter dans la conversation, pour limiter la taille du contexte — mais n'affecte pas le contenu du retrieveur, d'où la necessite de le tronquer separement.

Projet 30 — 30-compliance_agent

Analyse de conformite reglementaire UE, export Word

Objectif

Etant donne le nom d'une entreprise et son secteur d'activite, produire une analyse complete de conformite reglementaire europeenne : cartographie des reglements applicables (RGPD, Loi sur l'IA, Pacte vert, marquage CE...), evaluation des risques par criticite, et feuille de route de mise en conformite. Le rapport, redige en Markdown, est ensuite converti en document Word professionnel avec page de titre et pied de document.

Outils utilises

Outil	Role dans ce projet
Groq (openai/gpt-oss-120b)	Redige l'analyse de conformite.
ExaTools (optionnel)	Recherche web restreinte aux sources officielles UE (eur-lex.europa.eu, ec.europa.eu), activee seulement si une cle Exa est presente.
ReasoningTools	Aide l'agent a structurer son raisonnement avant de repondre.
python-docx	Convertit le rapport Markdown en document Word mis en forme (titres, listes, gras/italique).

Bugs trouves et corriges

- `show_tool_calls= obsolete` — renomme `debug_mode=`.
- **Pas de cle OpenAI** — `OpenAIChat(id='gpt-4-turbo')` remplace par Groq.
- `stream=True` **passe au constructeur de l'agent** cassait la recuperation du resultat : avec le streaming actif par default, `agent.run()` renvoie un generateur au lieu d'un objet unique, et `reponse.content` levait une erreur ('generator' object has no attribute 'content'). Corrige en retirant `stream=True` de la construction.
- **Crash d'encodage Windows** sur les emojis des messages de statut (,) — corrige avec la reconfiguration UTF-8 habituelle de `stdout/stderr`.
- **Etape de sauvegarde markdown manquante** : le code appelait `agent.print_response(requete)`, qui affiche la reponse dans la console mais ne la retourne ni ne la sauvegarde nulle part — la conversion Word qui suit cherchait un fichier `conformite_automobile.md` qui n'etait jamais cree. Corrige en utilisant `agent.run()` (qui retourne le contenu) et en l'ecrivant explicitement dans ce fichier.

Etat final : fonctionnel — rapport de conformite complet genere pour un cas test (Renault, industrie automobile) et exporte avec succes en document Word.

main.py

La classe AnalyseurConformiteUE : agent, generation, export Word.

Lignes 105-129 (corrige)

```
def analyser_entreprise(self, nom_entreprise, secteur,
                        contexte_supplementaire="",
                        chemin_markdown="conformite_automobile.md"):
    requete = f"""Effectuez une analyse complete de conformite reglementaire UE pour :
    Entreprise : {nom_entreprise}
    Secteur : {secteur}
    ..."""

    print(f"Debut de l'analyse de conformite pour {nom_entreprise}...")
    reponse = self.agent.run(requete)
    print(reponse.content)

    with open(chemin_markdown, "w", encoding="utf-8") as fichier:
        fichier.write(reponse.content)

    return reponse.content
```

Version corrigee : `agent.run()` (au lieu de `print_response()`) donne acces a `reponse.content`, qui est a la fois affiche ET sauvegarde dans le fichier markdown attendu par l'etape suivante — la ou le code original perdait silencieusement le contenu genere.

Lignes 179-207

```
def _traiter_contenu_markdown(self, doc: Document, contenu: str):
    lignes = contenu.splitlines()
    for ligne in lignes:
        ligne = ligne.strip()
        if not ligne:
            continue
        if ligne.startswith("# "):
            doc.add_heading(ligne[2:], level=1)
        elif ligne.startswith("## "):
            doc.add_heading(ligne[3:], level=2)
        elif ligne.startswith("** ") or ligne.startswith("- "):
            self._ajouter_puce(doc, ligne[2:].strip())
        elif re.match(r'^\d+\.', ligne):
            self._ajouter_numero(doc, re.sub(r'^\d+\.', '', ligne))
        else:
            self._ajouter_paragraphe_formate(doc, ligne)
```

Un petit convertisseur Markdown vers Word ecrit a la main, ligne par ligne : il reconnait les titres (#, ##...), les listes a puces et numerotees, et traite tout le reste comme un paragraphe normal. Le gras/italique (**texte**, *texte*) est gere separement par expressions regulieres dans `_ajouter_texte_formate`.

Projet 32 — 32-Advance-Research-agent

Assistant de recherche academique avec base de connaissances PgVector

Objectif

Uploader un article de recherche (PDF) via Streamlit ; le contenu est OCR'ise (Mistral), indexe dans une base de connaissances vectorielle (PgVector/Postgres), automatiquement resume, puis l'utilisateur peut poser des questions de suivi. L'agent de question/reponse cherche d'abord dans la base de connaissances (avec citation exacte de la source), puis se rabat sur Semantic Scholar ou une recherche web s'il ne trouve pas la reponse dans le document.

Outils utilises

Outil	Role dans ce projet
Streamlit	Upload du PDF, affichage du resume et du chat de questions/reponses.
Mistral OCR	Extrait le texte de l'article PDF en Markdown.
PgVector (Postgres)	Base de donnees vectorielle stockant les embeddings du document pour la recherche semantique.
sentence-transformers (local)	Genere les embeddings, sans dependre d'une API externe payante.
Groq (openai/gpt-oss-120b)	Agent de resume et agent de question/reponse.
ReasoningTools / DuckDuckGoTools	Raisonnement structure et recherche web de secours.
Semantic Scholar API	Recherche d'articles academiques externes en complement de la base de connaissances.

Bugs trouves et corriges

- **Import mistralai casse** — corrige comme dans les autres projets.
- **Cle API litteralement fausse** : le code contenait `OpenAIChat(id='gpt-4o-mini', api_key="openai_key")` — la chaine de caracteres litterale "openai_key" au lieu de la variable contenant la vraie cle. Corrige en passant a Groq de toute facon (pas de cle OpenAI disponible), ce qui elimine le bug par la meme occasion.
- **MarkdownKnowledgeBase n'existe plus** : remplacee par une nouvelle classe unifiee `agno.knowledge.knowledge.Knowledge`, dont le contenu se charge via `.add_content(path=...)` plutot qu'un argument de construction + `.load(recreate=...)`.
- **GoogleSearchTools disparu** — remplace par `DuckDuckGoTools`.

- **Base Postgres/pgvector absente** : le code pointait vers postgresql+psycopg://ai:ai@localhost:5532/ai, une base qui n'existait pas sur la machine. Une fois Docker réparé (voir 10-Workflow_Builder), un conteneur pgvector/pgvector:pg16 a été démarré avec exactement ces identifiants (`docker run -e POSTGRES_USER=ai -e POSTGRES_PASSWORD=ai -e POSTGRES_DB=ai -p 5532:5432 pgvector/pgvector:pg16`) et la connexion a été vérifiée avec succès.

Etat final : code entièrement corrigé, base Postgres/pgvector démarrée et connexion vérifiée — exécution complète non testée, faute de quota Groq disponible en fin de session.

research_assistant_agent.py

Setup de la base de connaissances et des 2 agents (resume, Q/R).

Lignes 59-68 (corrige)

```
def knowledge_base_setup():
    knowledge_base = Knowledge(
        vector_db=PgVector(
            table_name="markdown_documents",
            db_url="postgresql+psycopg://ai:ai@localhost:5532/ai",
            embedder=SentenceTransformerEmbedder(),
        ),
    )
    knowledge_base.add_content(path="DocumentMarkdown/ocr_document.md")
    return knowledge_base
```

L'embedder local (sentence-transformers) evite de dependre d'une cle API supplementaire juste pour les embeddings ; seul le modele de chat (Groq) et l'OCR (Mistral) restent des services externes.

Lignes 92-123

```
research_agent = Agent(
    name="Research Agent",
    model=Groq(id="openai/gpt-oss-120b", api_key=groq_key),
    instructions=dedent("""
1. Answering Questions: try to answer using the internal knowledge base first,
   cite the exact source (quote the relevant sentence or paragraph).
2. Fallback to External Tools: use semantic_scholar_search(query), then DuckDuckGo.
3. Always show the source link or citation.
"""),
    knowledge=knowledge_base,
    search_knowledge=True,
    tools=[ReasoningTools(add_instructions=True), semantic_scholar_search, DuckDuckGoTools()],
    compress_tool_results=True,
)
```

L'ordre de priorite est explicite dans les instructions : base de connaissances d'abord (avec citation exacte obligatoire), puis Semantic Scholar, puis recherche web generale en dernier recours — une hierarchie qui privilegie la fiabilite (source du document original) sur la disponibilite.

Projet 33 — 33-FreightAgent

Estimation de couts de fret (UPS/FedEx/DHL) via WhatsApp

Objectif

Bot WhatsApp permettant a une petite entreprise d'import/export d'estimer et comparer les couts d'expedition entre transporteurs (UPS, FedEx, DHL) : l'utilisateur envoie une description de colis (texte ou photo du produit), et l'agent cherche dans une base de connaissances construite a partir des grilles tarifaires 2025 des trois transporteurs (OCR'isees depuis leurs PDF), completee par une recherche web si necessaire.

Outils utilises

Outil	Role dans ce projet
FastAPI / uvicorn	Serveur webhook WhatsApp (identique en structure aux projets 16/23).
Mistral OCR	Convertit les 3 PDF de grilles tarifaires en Markdown.
LanceDb	Base de donnees vectorielle locale (fichier) indexant les grilles tarifaires.
sentence-transformers (local)	Embeddings locaux, gratuits et sans limite de debit.
Groq (openai/gpt-oss-20b)	Agent principal + agent de vision (decrit le produit a partir d'une photo).
DDGS	Recherche web complementaire (DuckDuckGo Search).

Bugs trouves et corriges

- **Import mistralai casse** — corrige comme dans les autres projets.
- **Pas de cle OpenAI** — OpenAIChat remplace par Groq.
- **MarkdownKnowledgeBase et agno.embedder.openai obsoletes** — migres vers la nouvelle classe Knowledge et un embedder local.
- **Rate-limit Mistral trop agressif pour les embeddings** : indexer 3 documents via l'API d'embeddings gratuite de Mistral declenchait des dizaines d'erreurs 429 en rafale. Corrige en basculant sur un embedder local (SentenceTransformerEmbedder), gratuit et sans limite de debit.
- **Table LanceDB perimee** : apres le changement d'embedder (1536 dimensions chez OpenAI -> 1024 chez Mistral -> 384 en local), l'ancienne table indexee avec une dimension differente provoquait une erreur de cast (Cannot cast to FixedSizeList(1536)). Corrige en supprimant le dossier tmp/lancedb pour forcer une reconstruction propre.
- **Depassement du quota Groq** malgre max_results=3 et compress_tool_results=True (jusqu'a ~15 000 tokens demandes) — passage au modele Groq 20B ; correctif partiellement suffisant, une requete a encore depasse la limite lors du dernier test.

Etat final : tous les bugs de compatibilite corriges ; une requete depasse encore occasionnellement la limite de 8000 tokens/minute de Groq malgre les optimisations — necessite un ajustement supplementaire (reduction du contexte de ReasoningTools ou des extraits de documents).

freight_agent.py

Base de connaissances locale (LanceDB) et l'agent de fret.

Lignes ~133-165 (corrige)

```
knowledge_base = Knowledge(
    vector_db=LanceDb(
        table_name="freight_documents",
        uri=LANCEDB_URI,
        search_type=SearchType.hybrid,
        embedder=SentenceTransformerEmbedder(),
    ),
    max_results=3,
)
if os.path.exists(MARKDOWN_DIR) and any(f.endswith(".md") for f in os.listdir(MARKDOWN_DIR)):
    knowledge_base.add_content(path=MARKDOWN_DIR)

freight_agent = Agent(
    name="Freight Agent",
    model=Groq(id="openai/gpt-oss-20b", api_key=groq_key),
    instructions=dedent("""..."""),
    tools=[ReasoningTools(add_instructions=True), ddgs_tool],
    knowledge=knowledge_base,
    search_knowledge=True,
    compress_tool_results=True,
)
```

search_type=SearchType.hybrid combine recherche par mots-cles et recherche semantique (vectorielle) dans LanceDB, ce qui donne generalement de meilleurs resultats qu'une recherche purement vectorielle sur des documents tres structures comme des grilles tarifaires (beaucoup de chiffres et de codes exacts a retrouver).

Projet 34 — 34-ImportExportSpecialist

Variante autonome (CLI) de conformité import/export, export Word direct

Objectif

Version consolidée et autonome du projet 27 : deux agents (conformité import/export via une base de connaissances LanceDB, puis réglementation locale via recherche web uniquement) produisent chacun un rapport HTML, combinés et convertis directement en document Word téléchargeable — sans nécessiter l'étape de traduction séparée du projet 27.

Outils utilisés

Outil	Rôle dans ce projet
Mistral OCR	OCR des lois UK/Maroc en Markdown (optionnel — désactive proprement si la clé manque).
LanceDb + Groq	Base de connaissances vectorielle des lois, et modèle de langage des 2 agents.
DuckDuckGoTools	Recherche web pour la réglementation locale du marché cible.
markdown2 / html2docx	Convertissent le HTML généré par les agents en document Word final.

Bugs trouvés et corrigés

- **Import mistralai, OpenAI->Groq, MarkdownKnowledgeBase->Knowledge** — mêmes corrections que les projets 27/32/33.
- **Crash d'encodage Windows** sur les émojis des messages de statut — même correctif UTF-8 habituel.

Etat final : fonctionnel — rapport de conformité complet généré et exporté en document Word avec succès (malgré quelques avertissements de recherche web sans résultat, gérés sans interrompre l'exécution).

full_import_export_agent.py

Setup de la base de connaissances (nouvelle API) et generation du rapport.

Lignes 59-70 (corrige)

```
def setup_knowledge_base():
    markdown_path = MARKDOWN_DIR
    if not os.path.exists(markdown_path):
        return None
    md_files = [f for f in os.listdir(markdown_path) if f.endswith('.md')]
    if not md_files:
        return None
    knowledge_base = Knowledge(
        vector_db=LanceDb(
            table_name="import_export_documents",
            uri=LANCEDB_URI,
            search_type=SearchType.hybrid,
            embedder=MistralEmbedder(api_key=mistral_key),
        ),
    )
    knowledge_base.add_content(path=markdown_path)
    return knowledge_base
```

Contrairement au projet 33, celui-ci n'indexe que 2 documents (pas 3), et le rate-limit Mistral sur les embeddings n'a pas pose de probleme lors du test — l'embedder Mistral a donc ete conserve ici plutot que remplace par un embedder local.

Lignes 260-289

```
def generate_report(product_description):
    output_html = ""
    agent1 = import_export_agent.run(product_description)
    output_html += agent1.content + "<hr>"
    agent2 = local_regulations_agent.run(product_description)
    output_html += agent2.content

    docx_bytes = html2docx(output_html, title="Import-Export Regulation Report")
    filename = "import_export_regulation_report.docx"
    with open(filename, "wb") as f:
        f.write(docx_bytes.getvalue())
    return filename
```

Les deux agents sont concus pour produire directement du **HTML** (impose dans leur expected_output) plutot que du Markdown : ce HTML est simplement concatene puis converti en un seul document Word via html2docx, une approche plus directe que le parseur Markdown ecrit a la main du projet 30.

Projet 14 — 14-Multi_AgentAGNO

Equipe d'agents finance (recherche web + donnees de marche) via Playground

Objectif

Fournir une equipe de 2 agents (recherche web et donnees financieres Yahoo Finance) coordonnee par un agent chef d'equipe qui produit des analyses d'investissement completes, avec memoire persistante des preferences utilisateur. Le tout est expose via l'interface web « Playground » integree a agno, accessible sur le port 7777.

Outils utilises

Outil	Role dans ce projet
<code>agno.team.Team</code> (coordinate)	Orchestre l'agent de recherche web et l'agent financier.
<code>OpenAIChat</code> (gpt-4o)	Modele des 2 agents et du chef d'equipe.
<code>DuckDuckGoTools</code> / <code>YFinanceTools</code>	Recherche web generale, et cours/ratios/recommandations Yahoo Finance.
<code>agno.memory.v2.Memory</code>	Memoire agentique persistante partagee entre les 2 agents et l'equipe.
<code>PostgresAgentStorage</code>	Persistence des sessions de conversation (connectee ici a MySQL, voir points d'attention).
<code>agno.playground.Playground</code>	Genere automatiquement une interface web de chat pour tester agents et equipe.

Variables d'environnement requises

- `OPENAI_API_KEY` (ou `OPENAI_KEY_API`) — obligatoire.
- `MYSQL_HOST`, `MYSQL_PORT`, `MYSQL_USER`, `MYSQL_PASSWORD`, `MYSQL_DATABASE` — connexion a un serveur MySQL local devant deja exister.

Points d'attention : les classes agno utilisees (`PostgresMemoryDb` / `PostgresAgentStorage`, malgre leur nom) sont connectees a une URL MySQL (`mysql+mysqlconnector://...`) plutot qu'a Postgres — incoherence de nommage qui fonctionne uniquement parce que ces classes reposent sur SQLAlchemy en interne. Le script exige un serveur MySQL prealablement installe et une base creee — bloque faute d'une telle infrastructure disponible.

`reasoning_finance_team.py`

Verification MySQL, memoire partagee, agents et Team.

Lignes ~30-45

```

def test_mysql_connection():
    try:
        engine = create_engine(db_url)
        with engine.connect() as connection:
            connection.execute(text("SELECT 1"))
            current_db = connection.execute(text("SELECT DATABASE()")).fetchone()[0]
            print(f"MySQL OK - Base active: {current_db}")
        return True
    except Exception as e:
        print(f"Erreur MySQL: {e}")
        return False

if not test_mysql_connection():
    exit(1)

```

Avant même de créer les agents, le script vérifie que la base MySQL est joignable via une requête triviale, et s'arrête immédiatement si ce n'est pas le cas — un échec explicite plutôt qu'un plantage tardif et confus au premier appel à la mémoire.

Lignes ~90-105

```

reasoning_finance_team = Team(
    name="Reasoning Finance Team",
    mode="coordinate",
    members=[web_agent, finance_agent],
    tools=[ReasoningTools(add_instructions=True)],
    storage=PostgresAgentStorage(db_url=db_url, table_name="reasoning_finance_team_sessions"),
    memory=memory,
    enable_agentic_context=True,
    enable_team_history=True,
    success_criteria="The team has provided a complete financial analysis with data, "
                    "visualizations, risk assessment, and actionable recommendations.",
)

```

En mode coordinate, l'équipe décide elle-même quels agents solliciter et dans quel ordre, avant de fusionner leurs réponses en une synthèse unique ; `success_criteria` sert de critère d'arrêt que l'équipe doit satisfaire avant de conclure.

Projet 16 — 16-Whatsapp

Bot WhatsApp gestionnaire de todo-list (Todoist)

Objectif

Permettre a un utilisateur de gerer sa liste de taches Todoist directement depuis WhatsApp, en langage naturel (« j'ai combien de taches demain ? », « ajoute une tache demain a 15h »). Chaîne complete : message WhatsApp -> webhook FastAPI -> agent IA (Agno + GPT-4o) qui interprete la demande et appelle les bonnes fonctions Todoist/date -> reponse renvoyee via l'API WhatsApp Business.

Outils utilises

Outil	Role dans ce projet
FastAPI / uvicorn	Serveur recevant les webhooks WhatsApp (verification GET, messages POST).
agno.agent.Agent (GPT-4o)	Interprete la demande et appelle les fonctions Todoist comme des outils.
API Todoist REST v2	Stockage et lecture des taches de l'utilisateur.
API WhatsApp Business Cloud (Meta)	Reception et envoi des messages WhatsApp.
timeapi.io	Fournit l'heure reelle actuelle (fuseau Africa/Casablanca), le LLM n'ayant pas d'horloge fiable.

Variables d'environnement requises

- OPENAI_API_KEY, TODOIST_API_KEY
- VERSION, PHONE_NUMBER_ID, WHATSAPP_ACCESS_TOKEN, VERIFY_TOKEN, RECIPIENT_PHONE_NUMBER

Points d'attention : necessite un compte WhatsApp Business Meta + un tunnel public (ngrok) pour recevoir les webhooks, et un compte Todoist — infrastructure externe non disponible. Aucune verification de signature de webhook (X-Hub-Signature-256) : n'importe qui connaissant l'URL pourrait forger des requetes. Le traitement suppose toujours un message texte (message['text']['body']) : une image/audio provoquerait une erreur non geree.

main.py / whatsapp_utils.py

Webhook FastAPI et traitement en arriere-plan.

Lignes ~

```
async def handle_message(request: Request):
    body = await request.json()
    if body.get("entry")[0].get("changes")[0].get("value", {}).get("statuses"):
        return JSONResponse(content={"status": "ok"})
    if is_valid_whatsapp_message(body):
        Thread(target=process_whatsapp_message, args=(body,)).start()
        return JSONResponse(content={"status": "ok"})
    return JSONResponse(status_code=404, content={"status": "error"})
```

Meta impose un delai de reponse court sur le webhook : le traitement reel (appel a l'agent IA, qui peut prendre plusieurs secondes) est donc lance dans un Thread separe, et le serveur repond immediatement « ok » pour eviter que Meta ne considere la requete en echec et ne la retente.

Lignes ~ (agent_service.py)

```
def get_tasks_for_date(date_str):
    all_tasks = get_tasks()
    return [
        {"content": t["content"], "time": t["due"].get("datetime", "Pas d'heure specifique")}
        for t in all_tasks
        if t.get("due") and t["due"].get("date") == date_str
    ]

agent = Agent(
    model=OpenAIChat(id="gpt-4o", api_key=OPENAI_API_KEY),
    tools=[get_current_datetime, get_today_date, get_tomorrow_date, get_tasks, get_tasks_for_date,
    add_task],
)
```

L'API Todoist ne permet pas de filtrer nativement par date exacte : le filtrage se fait donc cote client en Python. Toutes les fonctions utilitaires (dates, taches) sont donnees telles quelles a l'agent comme outils — c'est le modele qui decide, selon la question posee, lesquelles appeler et dans quel ordre.

Projet 18 — 18-Memory_agent

Assistants sante et immobilier avec memoire persistante (Mem0)

Objectif

Deux sous-projets independants demontrant l'usage de Mem0 (memoire semantique en tant que service) avec le framework Google ADK (Agent Development Kit) : un assistant de cabinet medical qui retient les informations partagees par un patient et peut programmer des rendez-vous, et un conseiller immobilier qui retient les preferences client et recherche des biens correspondants dans un catalogue local via recherche vectorielle FAISS.

Outils utilises

Outil	Role dans ce projet
<code>google.adk.agents.LlmAgent</code>	Framework Google Agent Development Kit pour construire l'agent conversationnel.
<code>LiteLlm</code>	Couche d'abstraction permettant a l'ADK (pense pour Gemini) d'utiliser un modele OpenAI (gpt-4o-mini).
<code>mem0.MemoryClient</code>	Service tiers de memoire vectorielle : stocke et recherche semantiquement les informations utilisateur.
<code>faiss</code> + <code>sentence-transformers</code>	(Immobilier uniquement) Indexation et recherche semantique locale du catalogue de biens.

Variables d'environnement requises

- `MEM0_API_KEY` — obligatoire pour l'immobilier (aucun repli) ; optionnelle pour la sante (repli automatique sur un dictionnaire Python local, non persistant, si absente).
- Une cle OpenAI est necessaire en pratique pour `LiteLlm`, meme si elle n'est pas explicitement lue dans le code du sous-projet sante.

Points d'attention : necessite un compte Mem0 (service tiers). Le sous-projet immobilier utilise un identifiant utilisateur (`USER_ID = "Ismo"`) code en dur — en l'etat, tous les clients partageraient la meme memoire, un probleme de confidentialite en production. `schedule_appointment` (sante) genere un identifiant via `hash(date+time)`, non stable entre executions Python. Aucun des deux fichiers n'a de boucle de conversation interactive reelle dans son bloc principal.

`Health-care/agent.py`

Service patient avec repli local si Mem0 est absent.

Lignes ~

```

class PatientService:
    def __init__(self, user_id):
        self.user_id = user_id
        if MEMO_API_KEY:
            self.memo_client = MemoryClient(api_key=MEMO_API_KEY)
            self.use_memo = True
        else:
            self.memo_client = None
            self.use_memo = False
            self.local_memory = {}

```

Mecanisme de repli explicite : sans cle MemO, le service utilise un simple dictionnaire Python en memoire locale au lieu du service externe, ce qui permet de tester l'agent sans compte MemO, au prix de perdre toute persistance entre redemarrages.

Real-Estate/agent.py.py

Recherche vectorielle FAISS locale sur le catalogue de biens.

Lignes ~

```

def _initialize_search_system(self):
    with open(os.path.join(BASE_DIR, "agency_property_data.json"), "r") as f:
        self.data = json.load(f)
    self.embed_model = SentenceTransformer("all-MiniLM-L6-v2")
    self.texts = self._extract_property_texts()
    embeddings = self.embed_model.encode(self.texts, convert_to_numpy=True)
    self.faiss_index = faiss.IndexFlatL2(embeddings.shape[1])
    self.faiss_index.add(embeddings)

```

Chaque bien immobilier (titre, description, prix, surface, chambres, equipements...) est aplati en une seule chaine de texte, puis vectorise localement (sans appel API) et indexe dans FAISS pour permettre une recherche semantique (« appartement 2 chambres avec balcon ») plutot qu'une recherche par mots-cles exacts.

Projet 23 — 23-Whatsapp-lette

Compagnon conversationnel WhatsApp avec support d'images (Letta)

Objectif

Tres proche du projet 16 dans son architecture (webhook FastAPI + API WhatsApp), mais remplace l'agent Agno/Todoist par un agent **Letta** (ex-MemGPT), specialise dans la memoire conversationnelle a tres long terme. Particularite : gere aussi bien le texte que les **images** envoyees par l'utilisateur (l'agent recoit alors un message multimodal image+legende).

Outils utilises

Outil	Role dans ce projet
FastAPI / uvicorn	Serveur webhook, identique en structure au projet 16.
letta_client.Letta	Client d'un agent Letta deja deploye (memoire persistante geree cote service).
API WhatsApp Business Cloud (Meta)	Reception/envoi de messages, et telechargement des images en 2 etapes (metadonnees puis binaire).

Variables d'environnement requises

- LETTA_API_KEY, AGENT_ID (agent Letta pre-cree, probablement via un template « Companion »)
- VERSION, PHONE_NUMBER_ID, WHATSAPP_ACCESS_TOKEN, VERIFY_TOKEN

Points d'attention : necessite un compte Letta + un agent deja deploye, en plus de l'infrastructure WhatsApp Business. Le dossier images/ n'est jamais cree automatiquement (pas de os.makedirs) : le telechargement d'image echouerait sans ce dossier pre-existant. Seuls les types text et image sont geres ; tout autre type (audio, document...) provoque une erreur non geree.

whatsapp_utils.py

Telechargement en 2 etapes d'une image WhatsApp entrante.

Lignes ~

```
elif message["type"] == "image":
    image_id = message["image"].get("id")
    metadata_res = requests.get(f"https://graph.facebook.com/{VERSION}/{image_id}",
    headers=headers)
    image_url = metadata_res.json().get("url")
    image_res = requests.get(image_url, headers=headers)
    with open(f"images/{image_id}.jpg", "wb") as f:
        f.write(image_res.content)
    message_body = {'image_path': f'images/{image_id}.jpg', 'caption':
    message["image"].get("caption", "")}
```

L'API WhatsApp ne transmet jamais le binaire de l'image directement dans l'evenement webhook : il faut d'abord echanger l'identifiant du media contre une URL de telechargement temporaire, puis telecharger reellement le fichier depuis cette URL — un processus en deux appels HTTP distincts.

Lignes ~ (agent_service.py)

```
if isinstance(message, dict):
    with open(message.get("image_path"), "rb") as image_file:
        encoded_string = base64.b64encode(image_file.read()).decode('utf-8')
    message = [
        {"type": "image", "source": {"type": "base64", "media_type": "image/jpeg", "data":
encoded_string}},
        {"type": "text", "text": message.get("caption")}
    ]
    answer = client.agents.messages.create(agent_id=AGENT_ID, messages=[{"role": "user", "content":
message}])
```

Quand le message est un dictionnaire (image), il est encode en base64 et transforme en message multimodal (bloc image + bloc texte) avant d'etre envoye a l'agent Letta — le meme point d'entree gere donc de maniere transparente le texte simple et les images legendees.

Projet 25 — 25-PriceAndPositioning

Analyse de prix et positionnement concurrentiel (recherche web IA)

Objectif

Aider a definir le bon prix et positionnement d'un nouveau produit en recherchant des produits concurrents reels (via Exa, un moteur de recherche semantique restreint a Amazon), en analysant leurs avis negatifs, puis en generant un rapport structure : fourchette de prix recommandee (Budget/Milieu de gamme/Premium), liste des concurrents, et recommandations tactiques.

Outils utilises

Outil	Role dans ce projet
<code>agno.workflow.Workflow</code>	Pipeline en 2 etapes : recherche (fonction Python) puis analyse (agent).
<code>exa_py.Exa</code>	Recherche web semantique restreinte aux domaines amazon.com/amazon.fr.
<code>openai.OpenAI</code> (client direct)	Nettoie/structure le contenu brut trouve par Exa en 3 produits avec avis.
Agent (OpenAIChat gpt-4o-mini)	Redige la strategie de prix finale a partir des donnees structurees.

Variables d'environnement requises

- OPENAI_API_KEY, EXA_API_KEY (mis de cote a la demande de l'utilisateur — non traite ce soir).

Points d'attention : le prompt systeme autorise explicitement le modele a inventer des avis negatifs plausibles si les vrais avis ne sont pas trouves dans le contenu recupere — un rapport genere pourrait donc melanger silencieusement donnees reelles et donnees fictives. En cas d'echec total d'Exa, un mode de repli genere une analyse de marche entierement fictive via le LLM seul, sans avertissement clair transmis a l'utilisateur final dans le rapport livre.

`price_and_positioning.py`

Recherche Exa puis structuration via un second appel LLM.

Lignes ~

```
completion = openai_client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "Analyse les resultats et extrait 3 produits pertinents avec... "
        "(pour les avis negatifs, cree des exemples realistes bases sur le type de produit si besoin)"},
        {"role": "user", "content": f"Analyse ces resultats pour {search_term}:"
        {formatted_content}}
    ],
    temperature=0.7
)
```

Le contenu brut recupere par Exa (souvent bruyant : pages HTML mal nettoyees) est passe a un second appel LLM independant de l'agent Agno, dont l'instruction autorise explicitement l'invention de contenu plausible en l'absence de vraies donnees — un choix qui privilegie la disponibilite d'un resultat exploitable a la stricte veracite des donnees.

Projet 26 — 26-PlateDetection

Detection de plaques d'immatriculation + alerte SMS

Objectif

Système de vidéosurveillance qui détecte les plaques d'immatriculation dans un flux vidéo, les compare à une base d'infractions connues, et envoie une alerte SMS automatique (Twilio) lorsqu'un véhicule recherché est repéré — pense pour un poste de police ou un contrôle routier automatisé.

Outils utilisés

Outil	Rôle dans ce projet
Streamlit	Interface de déclenchement du traitement vidéo.
Roboflow (InferencePipeline)	Pipeline de détection/OCR de plaques sur un flux vidéo, via un workflow Roboflow personnalisé.
agno.agent.Agent (GPT-4o)	Formate le message d'alerte à partir du JSON d'infraction et déclenche l'envoi du SMS.
Twilio	Service tiers d'envoi de SMS.

Variables d'environnement requises

- ROBOFLOW_API_KEY, TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_FROM_NUMBER, TWILIO_TO_NUMBER

Points d'attention : nécessite un workspace Roboflow spécifique déjà configuré (tp-qpwtk / custom-workflow), un compte Twilio, et un fichier vidéo local en dur (video/traffic-3.mp4, absent du depot). Le requirements.txt épingle des versions très anciennes (numpy==1.21.6, opencv-python-headless==4.6.0.66...) probablement incompatibles avec Python récent. La comparaison de plaques est une égalité stricte de chaînes, sans tolérance aux erreurs OCR.

plate_detect.py

Callback de traitement des frames vidéo et envoi d'alerte.

Lignes ~

```
def my_sink(result, video_frame):
    if video_frame.frame_id == MAX_FRAMES:
        pipeline.terminate()
        return
    frame_plates = {res['output'].strip() for group in result['open_ai'] for res in group if
res.get('output')}
    for plate in frame_plates:
        plate_info = plate_detection(plate)
        if plate_info and plate_info['license_plate'] not in DONE_PLATES:
            DONE_PLATES.append(plate_info['license_plate'])
            agent_response = agent.run(json.dumps(plate_info))
            ...
```

A chaque frame traitée par le pipeline Roboflow, les plaques détectées (issues du bloc OCR open_ai du workflow) sont dédoublées puis comparées à la base d'infractions locale ; toute correspondance non déjà signalée déclenche l'agent, qui rédige et envoie l'alerte SMS.

Projet 29 — 29-Investment_insight

Profil client + recommandation d'investissement

Objectif

Assistant pour gestionnaire de patrimoine : recherche le profil d'un client (base interne puis, en repli, recherche web), recupere des donnees financieres en temps reel (Yahoo Finance) sur un titre qui l'interesse, et redige un rapport de recommandation d'investissement structure (note, prix cible, secteur).

Outils utilises

Outil	Role dans ce projet
agno.team.Team	Coordonne un agent de recherche interne (fichier JSON) et un agent de recherche web, avec bascule conditionnelle.
ScrapeGraphTools ExaTools	/ Scraping intelligent de pages web et recherche semantique, pour le profil client externe.
YFinanceTools FileTools	/ Donnees financieres temps reel, et lecture/ecriture des rapports Markdown intermediaires.

Variables d'environnement requises

- OPENAI_API_KEY, SGAI_API_KEY (ScrapeGraphAI), EXA_API_KEY (mis de cote a la demande de l'utilisateur — non traite ce soir).

Points d'attention : le bloc d'execution principal n'enchaîne pas reellement les agents entre eux (chaque appel utilise un texte fixe en dur plutot que la sortie de l'agent precedent) — ce n'est donc pas un pipeline automatise fonctionnel bout-en-bout tel quel, plutot des demonstrations independantes. Deux agents differents portent le meme nom et ecrivent dans le meme fichier recommendation.md, avec risque d'ecrasement.

main.py

Equipe de recherche client avec delegation conditionnelle.

Lignes ~

```
client_research = Team(
    name="Client Research Agent",
    members=[internal_search, web_search],
    instructions="""
    1. First, delegate to the internal search agent.
    3. If it replies "CLIENT NOT FOUND", delegate to the web search agent.
    """,
)
```

La logique conditionnelle (« ne chercher sur le web que si le client n'est pas trouve en interne ») est purement declarative dans le prompt — rien dans le code Python ne force cette regle, elle depend entierement du bon respect des instructions par le modele et de l'orchestration du Team.

Projet 31 — 31-Procurement-agent

Recherche de fournisseurs pour une liste de produits

Objectif

Automatise une partie du sourcing fournisseurs : a partir d'une liste de produits et d'une localisation saisies interactivement, recherche des fournisseurs potentiels sur le web (Exa), puis un agent extrait et structure les donnees dans un fichier CSV en **ecrivant et executant lui-meme** un script Python, avant de rediger une synthese comparative avec recommandations.

Outils utilises

Outil	Role dans ce projet
<code>exa_py.Exa</code>	Recherche web semantique de fournisseurs/vendeurs.
<code>agno.workflow.Workflow</code>	Structure le pipeline recherche puis analyse.
<code>agno.tools.python.PythonTools</code>	Permet a l'agent d'ecrire et d'executer du code Python pour generer le CSV — pas de generation cote application.

Variables d'environnement requises

- `EXA_API_KEY` (verifiee explicitement au demarrage, arret sinon) — mis de cote a la demande de l'utilisateur.

Points d'attention : laisser l'agent generer et executer du code Python arbitraire via PythonTools est une fonctionnalite puissante mais risquee (pas de sandboxing visible). Une strategie de repli a deux niveaux gere les echecs de recherche Exa (recherche simplifiee, puis message d'erreur structure) — bonne pratique de robustesse.

`procurement_agent.py`

L'agent ecrit et execute lui-meme le script generant le CSV.

Lignes ~

```
procurement_agent = Agent(  
    model=OpenAIChat(id='gpt-4o'),  
    instructions="""  
    3. After extracting the data, write and execute a Python script that creates 'data.csv'  
        - Use the standard csv module, encoding='utf-8'  
        - Then use the PythonTools tool to run the script  
    """,  
    tools=[PythonTools()],  
)
```

Plutot que d'ecrire lui-meme le CSV via une fonction Python fixe (comme dans d'autres projets du depot), l'agent est instruit d'ecrire et d'executer un script generique via l'outil PythonTools — approche plus flexible (colonnes/valeurs decidees dynamiquement) mais qui revient a laisser le LLM executer du code local arbitraire.

Projet 35 — 35-HR-Assistant

Assistant RH (conges, projets, approbations) via Airtable

Objectif

Assistant RH conversationnel pour les employes : consulter son solde de conges, faire une demande de conges (avec approbation par email par les RH via une page web dediee), et consulter ses informations de projet. Architecture en 2 parties : un backend Flask (agents agno + base Airtable + tunnel ngrok public) et un frontend Streamlit separe qui interroge ce backend.

Outils utilises

Outil	Role dans ce projet
agno.team.Team	Route chaque requete vers l'un de 3 agents specialises (conges, projets, approbation).
Flask + flask_cors	Serveur web exposant /chat, /text, /request/<id>, /process/<id>.
pyairtable	Base de donnees RH (conges, projets, demandes en attente).
pyngrok	Tunnel public temporaire pour que les liens d'approbation par email soient accessibles hors reseau local.
agno.tools.email.EmailTools	Envoi d'emails via Gmail (necessite client_secret.json, desactive proprement sinon).
Retell AI (appels REST directs)	Canal vocal alternatif via l'endpoint /text, independant des agents agno.

Variables d'environnement requises

- OPENAI_API_KEY, AIRTABLE_TOKEN, AIRTABLE_BASE_ID, NGROK_AUTH_TOKEN, HR_EMAIL — toutes validees explicitement au demarrage (arret propre si l'une manque).

SECURITE — Cle API tierce codee en clair dans le code source. Dans rhAgent.py, la fonction voice_handler (endpoint /text) contient litteralement : Authorization: Bearer key_1581ac824f79644c48f3f9c8fc51 — a deux reprises, alors que toutes les autres cles du projet sont correctement chargees depuis .env. Cette cle Retell AI (et l'agent_id associe) sont donc exposes a quiconque a acces au code source. Recommandation : revoquer et regenerer cette cle cote Retell AI immediatement, puis la migrer vers une variable d'environnement.

rhAgent.py

Cle API codee en dur (a corriger) et flux d'approbation humain-dans-la-boucle.

[Lignes extrait reel du fichier](#)

```
headers={
    "Authorization": f"Bearer key_1581ac824f79644c48f3f9c8fc51",
    "Content-Type": "application/json"
},
json={"agent_id": "agent_4cb8cf66474b91a09d19419", "metadata": {}},
...
RETELL_API_KEY = "key_1581ac824f79644c48f3f9c8fc51" # (corrige)
```

Le commentaire « (corrige) » suggère qu'un développeur a déjà tenté de « corriger » un problème en dupliquant la clé en dur plutôt que de l'externaliser correctement — la vraie correction est de la charger via `os.getenv("RETELL_API_KEY")` comme le reste des secrets du projet.

Lignes ~

```
@app.route('/process/<request_id>', methods=['POST'])
def process_request(request_id):
    action = request.form.get('action')
    record = request_table.all(formula=f"{{{request_id}}}='{{{request_id}}}'")[0]
    request_table.update(record['id'], {'status': action})
    approval_agent.run(f"Process holiday request {request_id} with action {action}",
    markdown=True)
    return f"Request {action} successfully. The employee will be notified."
```

Bon exemple d'humain-dans-la-boucle : un email envoyé aux RH contient un lien vers une page d'approbation web ; le clic sur Approve/Disapprove met à jour Airtable puis délègue à l'agent d'approbation le soin d'en tirer les conséquences (mise à jour du solde, email de confirmation à l'employé).

Projet 36 — 36-rfp-Agent

Remplissage automatique d'un questionnaire d'appel d'offres (Google Sheets)

Objectif

Automatise le remplissage de questionnaires RFP (appels d'offres) recus sous forme de Google Sheets. A partir d'une base de connaissances interne (documents Markdown decrivant l'entreprise, produits, tarification), un agent lit la feuille partiellement remplie, recherche les reponses pertinentes, et reecrit la feuille avec les champs completes.

Outils utilises

Outil	Role dans ce projet
<code>agno.knowledge.knowledge.Knowledge + Qdrant</code>	Base de connaissances vectorielle des documents internes de l'entreprise.
<code>pygsheets</code>	Lecture/ecriture de la feuille Google Sheets via un compte de service.
<code>ReasoningTools</code>	Raisonnement structure avant de remplir chaque champ.
3 outils personnalisés (@tool)	<code>list_worksheets</code> , <code>read_sheet_as_df</code> , <code>write_df_to_sheet</code> — donnent a l'agent un acces complet a la feuille.

Variables d'environnement requises

- `QDRANT_API_KEY`, `QDRANT_URL`, `OPENAI_API_KEY`
- Un fichier `client_secret.json` (compte de service Google) doit exister — la feuille cible doit aussi etre partagee avec l'email de ce compte de service.

Points d'attention : l'identifiant de feuille Google Sheets cible est code en dur dans le script (pas de saisie interactive). L'ecriture efface entierement l'onglet avant de le reecrire (`wks.clear()`) : si la reponse de l'agent ne contient pas toutes les colonnes d'origine, il y a un risque reel de perte de donnees existantes. Les fichiers Markdown de la base de connaissances fournis contiennent encore des placeholders non remplis ([Votre Nom d'Entreprise]) — a completer avant usage reel.

`main.py`

Outil d'ecriture tolerant aux reponses JSON mal formees du LLM.

Lignes ~

```
@tool(name="write_df_to_sheet", ...)
def write_df_to_sheet(df_json: str, sheet_id: str, worksheet_title: str, start_cell=[1, 1]) ->
str:
    if df_json.strip().startswith("```json"):
        df_json = df_json.replace("```json", "").replace("```", "").strip()
    df = pd.read_json(StringIO(df_json))
    wks = sh.worksheet_by_title(worksheet_title)
    wks.clear()
    wks.set_dataframe(df, start=tuple(start_cell))
```

L'outil nettoie d'abord les balises de bloc de code Markdown qu'un LLM ajoute fréquemment autour de sa sortie JSON, avant de la parser — une tolérance nécessaire car le modèle ne respecte pas toujours exactement le format demandé.